

CosineAnnealingWarmRestarts#

https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingWarmRestarts.html

Description:

Set the learning rate of each parameter group using a cosine annealing schedule. The $\eta_{\max}$ is set to the initial lr, T_{cur} is the number of epochs since the last restart and T_i is the number of epochs between two warm restarts in SGDR: When $T_{\text{cur}} = T_i$, set $\eta_t = \eta_{\min}$. When $T_{\text{cur}} = 0$ after restart, set $\eta_t = \eta_{\max}$. It has been proposed in SGDR: Stochastic Gradient Descent with Warm Restarts. optimizer (Optimizer) – Wrapped optimizer.

Function Signature

```
class  
torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer, T_0, T_mult=1, eta_min=0.0, last_epoch=-1)[source]#
```

Parameters

```
get_last_lr()[source]#
```

Return type

```
get_lr()[source]#
```

Returns:

```
dict[str, Any]
```

Code Examples

```
>>> optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  
>>> scheduler =  
torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(  
...     optimizer, T_0=20  
... )  
>>> for epoch in range(100):  
>>>     train(...)  
>>>     validate(...)  
>>>     scheduler.step()
```

```

>>> scheduler = CosineAnnealingWarmRestarts(optimizer, T_0,
T_mult)
>>> iters = len(dataloader)
>>> for epoch in range(20):
>>>     for i, sample in enumerate(dataloader):
>>>         inputs, labels = sample['inputs'], sample['labels']
>>>         optimizer.zero_grad()
>>>         outputs = net(inputs)
>>>         loss = criterion(outputs, labels)
>>>         loss.backward()
>>>         optimizer.step()
>>>         scheduler.step(epoch + i / iters)

```

```

>>> scheduler = CosineAnnealingWarmRestarts(optimizer, T_0,
T_mult)
>>> for epoch in range(20):
>>>     scheduler.step()
>>> scheduler.step(26)
>>> scheduler.step() # scheduler.step(27), instead of
scheduler(20)

```

Detailed Documentation

Documentation

Set the learning rate of each parameter group using a cosine annealing schedule.

The $\eta_{\max}$ is set to the initial lr, T_{cur} is the number of epochs since the last restart and T_i is the number of epochs between two warm

restarts in SGDR:

When $T_{\text{cur}} = T_i$, set $\eta_t = \eta_{\min}$. When $T_{\text{cur}} = 0$ after restart, set $\eta_t = \eta_{\max}$.

It has been proposed in SGDR: Stochastic Gradient Descent with Warm Restarts.

optimizer (Optimizer) – Wrapped optimizer.

T_0 (int) – Number of iterations until the first restart.

T_{mult} (int, optional) – A factor by which T_i increases after a restart. Default: 1.

$\eta_{\min}$ (float, optional) – Minimum learning rate. Default: 0.

last_epoch (int, optional) – The index of the last epoch. Default: -1.

Return last computed learning rate by current scheduler.

Compute the initial learning rate.

Load the scheduler's state.

state_dict (dict) – scheduler state. Should be an object returned from a call to state_dict().

Return the state of the scheduler as a dict.

It contains an entry for every variable in self.__dict__ which is not the optimizer.

Step could be called after every batch update.

This function can be called in an interleaved way.

